The broker can also be seen as an alerting system that triggers conversations between consumers and providers when there are failures.

A real example illustrated in figure 3.4 shows how different versions of the API are used by different consumers at a large online fashion retailer. Both consumers were using an API endpoint that returned customer data. The mobile application consumes version 2 of the API, whereas the legacy desktop application still uses version 1 of the API. The customer support team responsible for the legacy desktop application had a release cycle that was six months long, and they had never prioritized migrating to version 2 of the customer data API service. The provider team had one core model, shared between version 1 and version 2, to avoid duplicate code and reduce the development time of updating two versions of the same API. When they identified that they were getting bad data into the customer database, they wanted to stop this by changing one of the fields to mandatory. The code changes were made, and tests within the API project now conform to the new mandatory field. However, the legacy desktop application wasn't available in the test environment as the integration wasn't working, and the release date was a couple of months away, so no one was looking into it. The legacy desktop application started erroring when releasing the API in production with the core model updates. Due to the API now requiring a mandatory field, their `PUT` request failed because it didn't contain this data. Due to the six-month release cycle of the legacy desktop application, the API team had to revert the changes and only change version 2.